

Trabalho 3 de Fundamentos da Programação

Autores: Gabriel Carivalis, Dagno Camargo e Carolina Bruno

RAs: 2882019, 2877503, 2812100

Com a orientação dos professores entendemos o trabalho e começamos a pensar em uma lógica para a volta do robô para a primeira posição após ter encontrado a água. Após algumas tentativas, pesquisas na internet e orientações dos professores decidimos aplicar um algoritmo chamado “Breadth-First Search” (BFS). Cada participante em sua casa, após a aula do dia 13 de junho, primeiro dia que começamos a construir o código, começou a contribuir com o trabalho. Carolina e Dagno construíram de maneira colaborativa o algoritmo que fazia com que o robô voltasse para a entrada da caverna.

As principais dificuldades para criar o algoritmos que fazia com que o robô retornava foram: organizar e elaborar a forma como o robô se movia ao voltar; encontrar uma maneira de analisar as casas adjacentes ao robô, entender como funcionava os arquivos.c e .h do trabalho, a divisão do trabalho em funções e, por fim, entender o algoritmo BFS e aplicá-lo.

Num primeiro momento, houve o problema da abstração de enxergar a movimentação e analisar as casas adjacentes. Isso porque um objeto se deslocando por uma matriz e a análise das posições ainda parecia uma ideia abstrata, mas isso pôde ser superado com o contato e familiarização dessas ideias. A dificuldade de entender como funcionava o arquivo .c e .h foi superada estudando brevemente o material de TAD. Entender o funcionamento do algoritmo BFS foi um desafio interessante pois compreender a ideia de resolução de problema de outra pessoa e aplicá-la foi algo um pouco diferente do que estávamos fazendo até agora, pois, até então, criamos e desenvolvemos o nosso próprio modelo do zero. A dificuldade da modularização foi superada a partir de discussões entre os membros.

No código da ida código da ida, o robô foi dos 18% iniciais, com o código da volta + a ida totalmente aleatória, para os 67%, após vários testes. A escolha do algoritmo foi simples, pois resolvemos utilizar um algoritmo de busca em profundidade (DFS). Ele possui uma lógica simples de ser entendida, e não seria tão difícil de ser implementada, se não fosse pela possibilidade das lavas, o que dificultou e muito o processo para obter essa porcentagem inicial. Num primeiro momento, o código apenas detectava se era seguro ou se teria a possibilidade de morrer, mas estava muito básico. Após alguns aperfeiçoamentos e melhor clareza sobre as ideias, pensamos numa maneira de deduzir as lavas, subindo bastante a porcentagem, para mais de 80%. Outra coisa observada foi que há ordem com que ele se movia não poderia ser aleatória, priorizando assim uma ordem “para dentro” do tabuleiro, com as melhores taxas de porcentagem sendo de “cima” e “esquerda” para depois ir para “baixo” e “direita”. Após isso, e mais alguns ajustes pontuais, conseguimos finalmente passar dos 90% (já quase desistindo e querendo deixar os 80%). Foi considerado, durante o protótipo do código, usar outros algoritmos, como o dijkstra (achamos de elevada complexidade para implementação, por isso resolvemos alterar). Também descobrimos sobre o jogo “Hunt the Wumpus”, o qual provavelmente serviu de inspiração para o trabalho junto do campo minado, mas ele é focado para o desenvolvimento de IAs.